

初赛备赛 完整版

模块一 计算机系统与发展历史

作战目标： 了解我们手中的“武器”——计算机，是如何诞生和发展的。这部分知识就像了解历史战役一样，能帮助我们更好地理解计算机的“脾气”和工作原理。

1. 信息科学与计算机史要点

1.1 图灵奖 (A.M. Turing Award)

- 这是什么？
 - 想象一下，在电影界有奥斯卡金像奖，在科学界有诺贝尔奖。那么在计算机科学这个神奇的世界里，最高荣誉的奖项就是**图灵奖**！
 - 它由一个叫做“美国计算机学会”（ACM）的组织颁发，每年只发给一两个人，奖励那些对计算机科学做出了像发明电灯一样伟大贡献的科学家。
 - 这个奖项的名字是为了纪念一位伟大的计算机科学之父——**艾伦·图灵**。
- 著名获奖者（我们需要认识的英雄）：
 - **艾伦·图灵 (Alan Turing)**：虽然图灵奖是为纪念他而设立的，但他本人并没有获得过这个奖项。他是计算机科学和人工智能的奠基人，提出了“图灵机”这个理论模型，可以看作是所有现代计算机的理论祖先。
 - **约翰·冯·诺依曼 (John von Neumann)**：这位英雄提出了一个革命性的想法——“存储程序”，就是我们现在计算机工作的基本原理。我们后面会详细学习这个“冯·诺依曼体系结构”。
 - **姚期智 (Andrew Chi-Chih Yao)**：这是我们华人的骄傲！他在2000年获得了图灵奖，是目前唯一一位获得此奖的华人科学家。他的研究让我们的网络通信和密码变得更加安全。

1.2 计算机先驱（开创时代的伟人）

- **艾伦·图灵 (Alan Turing)**：
 - **图灵机模型**：他想象出一种非常简单的机器，只需要纸带、读写头和一套规则，就能模拟所有复杂的计算。这证明了“计算”这件事是可以被机器完成的。
 - **图灵测试**：他提出了一个判断机器是否具有智能的方法：让一个人和一台机器隔着屏幕与你聊天，如果你分不清哪个是人，哪个是机器，那么这台机器就通过了图灵测试。
- **约翰·冯·诺依曼 (John von Neumann)**：
 - **“存储程序”思想**：在他之前，计算机每次执行新任务，都需要像重新插拔电线一样修改硬件，非常麻烦。冯·诺依曼提出，可以把指挥计算机的“指令”（程序）和要处理的“数字”（数据）都存放在同一个地方（内存），计算机可以自己按顺序读取指令来执行。这个思想是现代计算机的基石！

1.3 计算机历史里程碑

- **ENIAC (埃尼阿克, 1946年):**
 - 世界上第一台通用电子计算机。
 - 有多大？想象一下，它像一间大教室那么大，重得像几头大象（超过30吨），工作起来发热巨大，需要强大的冷却系统。
 - 核心元件：它使用了约18000个**电子管**。电子管就像一个特殊的灯泡，可以控制电流的通断，但它又大又耗电，还容易坏。
 - 怎么编程？当时的程序员需要像接线员一样，手动插拔大量的电线和设置开关来告诉它做什么，非常原始。
- **冯·诺依曼体系结构 (1945年首次在报告中提出):**
 - 这是所有现代计算机的“设计图纸”。它规定计算机必须由五个部分组成：
 1. **输入设备**：告诉计算机要做什么。比如：键盘、鼠标。
 2. **输出设备**：让计算机告诉我们结果。比如：显示器、打印机。
 3. **存储器**：存放程序和数据的地方。就像我们的大脑，用来记忆。
 4. **运算器 (ALU)**：负责进行数学计算（加减乘除）和逻辑判断（是真是假）。就像我们的计算能力。
 5. **控制器 (CU)**：指挥各个部分协调工作的大脑中枢。它负责读取指令，并告诉其他部件该干什么。
 - **核心思想：存储程序**。指令和数据都以二进制码的形式存放在存储器中。

1.4 计算机的“代”

就像人类会经历童年、少年、青年、壮年一样，计算机也根据其核心电子元件的不同，经历了四个“世代”的进化。

代别	年代	核心元件	特点
第一代	1946–1958	电子管	体积巨大、耗电、昂贵、易坏
第二代	1959–1964	晶体管	体积变小、功耗降低、更可靠
第三代	1965–1970	集成电路 (IC)	将很多晶体管集成在一块小芯片上，体积更小，性能更强
第四代	1971–至今	大规模/超大规模集成电路 (LSI/VLSI)	在指甲盖大小的芯片上集成了数百万甚至数十亿个晶体管，催生了个人电脑和智能手机

高频考点： 记住每一代的核心元件是初赛选择题的热门考点！

1.5 按性能和规模分类

- **超级计算机 (巨型机):** 运算速度最快的计算机，用于解决“国之重器”级别的难题，比如天气预报、模拟核试验、新药研发等。中国的“神威·太湖之光”和“天河”系列都是著名的超级计算机。

- **大型机**：通常用于银行、航空公司、政府等需要同时处理大量用户请求的场景，稳定性和安全性极高。
- **小型机/工作站**：性能强大的个人电脑，主要用于科学和工程领域，如动画渲染、3D建模。
- **微型机 (PC)**：就是我们日常使用的个人电脑、笔记本电脑、智能手机、平板电脑等。

2. 计算机应用领域

计算机已经渗透到我们生活的方方面面，主要应用在以下几个领域：

- **科学计算**：进行复杂的数学和工程计算，比如计算火箭的飞行轨道。
- **信息处理**：对大量数据进行管理，如图书管理系统、学籍管理系统。
- **自动控制**：在工业生产线、无人驾驶汽车中，计算机实时监测环境并做出控制决策。
- **计算机辅助技术**：
 - **CAD (Computer-Aided Design)**：计算机辅助设计，如用电脑画建筑图纸。
 - **CAM (Computer-Aided Manufacturing)**：计算机辅助制造，如控制机器臂进行生产。
- **人工智能 (AI)**：让计算机模仿人的智能，如下棋的AlphaGo、人脸识别、语音助手等。
- **网络与通信**：上网、聊天、看视频等，都离不开计算机网络。

模块二 数据表示与运算

作战目标： 揭开计算机内部世界的秘密。计算机并不认识数字、文字和图片，它只认识0和1。我们将学习如何把我们世界的信息翻译成计算机的语言。

1. 存储单位与进制

1.1 单位换算（竞赛中请严格遵守这个标准）

- **比特 (bit)**：计算机中最小的信息单位，只能表示0或1。
- **字节 (Byte)**：最常用的基本单位，1个字节由8个比特组成。

- $$1 \text{ Byte} = 8 \text{ bit}$$

- **KB, MB, GB, TB...**：
 - 在计算机竞赛和操作系统中，换算关系是

$$2^{10}$$

，也就是1024。

- $$1 \text{ KB} = 1024 \text{ B} = 2^{10} \text{ B}$$

- $$1 \text{ MB} = 1024 \text{ KB} = 2^{20} \text{ B}$$

- $1 \text{ GB} = 1024 \text{ MB} = 2^{30} \text{ B}$
- $1 \text{ TB} = 1024 \text{ GB} = 2^{40} \text{ B}$

注意：硬盘厂商通常使用1000作为换算单位，所以你买的512GB的硬盘，在电脑上显示会比512GB小一些，这是正常的。但在做题时，**一律使用1024！**

1.2 图片存储量计算

- **核心概念：**图片是由一个个微小的颜色点组成的，这些点叫做“像素”。一张图片的大小，取决于它有多少像素，以及每个像素的颜色有多丰富。
- **色深 (Color Depth)：**表示一个像素点可以用多少个比特来存储颜色信息。
 - **24位真彩色 (24 bit)：**这是最常见的色深，意味着每个像素点用24个比特来表示颜色。因为

$$2^{24}$$

大约是1677万，所以可以显示非常丰富的色彩。

- **计算公式：**

$$\text{存储量(Byte)} = \frac{\text{图片宽度(像素)} \times \text{图片高度(像素)} \times \text{色深(bit)}}{8}$$

- **文字模拟例子：**
 - **问题：**一张分辨率为 $1024 * 768$ ，颜色为24位真彩色的图片，未经压缩时占用多少MB的存储空间？
 - **分析：**

1. **总像素数：**

$$1024 \times 768$$

2. **每个像素占用的比特数：** 24 bit

3. **总比特数：**

$$1024 \times 768 \times 24$$

4. **换算成字节(Byte)：**除以8。

$$\frac{1024 \times 768 \times 24}{8} = 1024 \times 768 \times 3 \text{ B}$$

5. **换算成KB：**再除以1024。

$$\frac{1024 \times 768 \times 3}{1024} = 768 \times 3 = 2304 \text{ KB}$$

6. **换算成MB：**再除以1024。

$$\frac{2304}{1024} = 2.25 \text{ MB}$$

- 代码演示（概念性）

```
#include <iostream>

int main() {
    // 定义图片的宽度、高度和色深
    int width = 1024; // 宽度为1024像素
    int height = 768; // 高度为768像素
    int depth = 24;    // 色深为24位

    // 计算总字节数
    // 注意：这里使用 long long 是为了防止中间结果溢出，实际考试中用笔算
    long long total_bits = (long long)width * height * depth;
    long long total_bytes = total_bits / 8;

    // 换算成MB
    double total_mb = (double)total_bytes / 1024 / 1024;

    // 输出结果
    // std::cout 是C++中用于输出的工具
    std::cout << "图片存储空间约为: " << total_mb << " MB" << std::endl;

    return 0;
}
```

1.3 视频存储量计算（了解即可）

视频可以看作是连续播放的图片。所以计算视频大小，就是在图片的基础上，乘以“帧率”和“时长”。

- 公式：

$$\text{存储量(Byte)} = \frac{\text{分辨率(宽} \times \text{高)} \times \text{帧率(fps)} \times \text{时长(s)} \times \text{色深(bit)}}{8}$$

- 这个计算出来的是未经压缩的视频大小，通常会非常巨大，所以实际视频都会经过压缩。

2. 进制与转换

计算机只懂二进制（逢二进一），我们习惯用十进制（逢十进一）。为了方便，程序员还常用八进制（逢八进一）和十六进制（逢十六进一）。

- 十六进制：用0-9和A-F表示。A代表10, B代表11, ..., F代表15。

2.1 任意进制转十进制（按权展开法）

- 方法：将每一位的数字乘以其对应的“位权”，然后相加。位权就是进制的（位数-1）次方。
- 文字模拟例子：

- 问题：将二进制数

$$(1101.1)_2$$

转换为十进制。

- 分析：

- 整数部分：

$$1101$$

- 第1位（从右往左数）是1，位权是

$$2^0 = 1$$

，值为

$$1 \times 1 = 1$$

- 第2位是0，位权是

$$2^1 = 2$$

，值为

$$0 \times 2 = 0$$

- 第3位是1，位权是

$$2^2 = 4$$

，值为

$$1 \times 4 = 4$$

- 第4位是1，位权是

$$2^3 = 8$$

，值为

$$1 \times 8 = 8$$

- 小数部分：

$$.1$$

- 小数点后第1位是1，位权是

$$2^{-1} = 0.5$$

，值为

$$1 \times 0.5 = 0.5$$

- 总和：

$$8 + 4 + 0 + 1 + 0.5 = 13.5$$

- 所以,

$$(1101.1)_2 = (13.5)_{10}$$

2.2 十进制转任意进制（整数除N取余，小数乘N取整）

- 整数部分：除N取余，逆序排列
 - 文字模拟例子：将十进制数

$$(25)_{10}$$

转换为二进制。

- 分析：

$$1. \quad 25 \div 2 = 12$$

..... 余 **1** (最低位)

$$2. \quad 12 \div 2 = 6$$

..... 余 **0**

$$3. \quad 6 \div 2 = 3$$

..... 余 **0**

$$4. \quad 3 \div 2 = 1$$

..... 余 **1**

$$5. \quad 1 \div 2 = 0$$

..... 余 **1** (最高位)

- 结果：将余数从下往上读，得到

$$(11001)_2$$

。

- 小数部分：乘N取整，顺序排列
 - 文字模拟例子：将十进制数

$$(0.8125)_{10}$$

转换为二进制。

- 分析：

$$1. \quad 0.8125 \times 2 = 1.625$$

..... 取整数 **1** (最高位)

$$2. \quad 0.625 \times 2 = 1.25$$

..... 取整数 1

3. $0.25 \times 2 = 0.5$

..... 取整数 0

4. $0.5 \times 2 = 1.0$

..... 取整数 1 (最低位)

- 结果：将整数部分从上往下读，得到

$$(0.1101)_2$$

。

3. 负数表示：原码、反码、补码

背景：计算机的运算器只会做加法。为了把减法也变成加法（比如

$$5 - 3$$

变成

$$5 + (-3)$$

)，科学家发明了补码。

我们以一个8位的 char 类型为例，最高位是**符号位**（0代表正数，1代表负数）。

- 原码：最直观的表达法。
 - 正数：符号位为0，其余位是数值的二进制。

- $[+5]_{\text{原}} = 00000101$

- 负数：符号位为1，其余位是数值的二进制。

- $[-5]_{\text{原}} = 10000101$

- 问题：0的表示不唯一（

$$+0$$

和

$$-0$$

)，且原码直接相加减法会出错。

- 反码：
 - 正数：反码与原码相同。

- $[+5]_{\text{反}} = 00000101$

- 负数：符号位不变，数值位按位取反（0变1，1变0）。

- $[-5]_{\text{原}} = 10000101 \rightarrow [-5]_{\text{反}} = 11111010$

- **补码**：计算机实际存储和运算使用的编码。

- **正数**：补码与原码、反码都相同。

- $[+5]_{\text{补}} = 00000101$

- **负数**：在其反码的末尾加1。

- $[-5]_{\text{反}} = 11111010 \rightarrow [-5]_{\text{补}} = 11111011$

- **优点**：

1. 0的表示唯一：

$$[+0]_{\text{补}} = [-0]_{\text{补}} = 00000000$$

。

2. 可以将减法统一为加法运算。例如计算

$$3 - 5$$

：

$$\text{补} + [-5]_{\text{补}} = 00000011 + 11111011 = (1)11111110$$

最高位的进位1被舍弃，结果是

$$11111110$$

。这个补码对应的十进制数就是-2，计算正确！

高频考点：给定一个负数，求其补码；或者给定一个补码，求其对应的十进制数。

4. 字符编码

计算机不仅要处理数字，还要处理文字。字符编码就是一张“密码本”，规定了哪个数字对应哪个字符。

4.1 ASCII码

- **定义**：美国标准信息交换代码，是计算机中最基础的字符编码。它用7个比特（后来扩展为8位，即一个字节）来表示128个常用字符。
- **必记规律**：
 - **数字**：'0' (48) 到 '9' (57)
 - **大写字母**：'A' (65) 到 'Z' (90)
 - **小写字母**：'a' (97) 到 'z' (122)
 - **关系**：同一个字母的大小写，其ASCII码值相差32。例如 'a' - 'A' = 97 - 65 = 32。
 - **空格**：32
 - **换行**：10

4.2 汉字编码

- **GB2312**：中国的国家标准编码，用两个字节（16位）来表示一个汉字。
- **汉字字形存储（点阵）**：
 - 为了在屏幕上显示汉字，需要用点阵来描述汉字的形状。
 - 一个

$$16 \times 16$$

的点阵，就像在一个16行16列的格子里，用黑点和白点画出汉字。

- **存储计算**：总共需要

$$16 \times 16 = 256$$

个点。每个点用1个比特（0或1）来表示，所以一个汉字字形需要256个比特。

- $256 \text{ bit} \div 8 = 32 \text{ Byte}$
- **高频考点**：计算存储100个

$$16 \times 16$$

点阵的汉字需要多少存储空间。

- $100 \times 32 \text{ B} = 3200 \text{ B} \approx 3.125 \text{ KB}$
- **Unicode / UTF-8**：
 - **Unicode**：为了统一全世界所有语言的编码，Unicode诞生了。它是一个巨大的字符集，包含了几乎所有国家的文字。
 - **UTF-8**：是Unicode的一种实现方式。它是一种**变长编码**，可以用1到4个字节来表示一个字符。
 - 对于英文字母，UTF-8只用1个字节，与ASCII码完全兼容。
 - 对于汉字，UTF-8通常用3个字节。
 - 这使得UTF-8在存储英文和中文混合的文本时非常高效。

模块三 硬件系统核心

作战目标：深入计算机的“五脏六腑”，理解CPU、内存、硬盘等核心部件是如何协同工作的。这有助于我们写出更高效、更健壮的程序。

1. CPU 架构详解 (中央处理器)

CPU是计算机的大脑，负责执行指令和处理数据。它主要由三部分构成。

1.1 三大核心部件

- **运算器 (ALU - Arithmetic Logic Unit)**：

- **功能：**执行所有算术运算（加、减、乘、除）和逻辑运算（与、或、非、异或）。程序中所有的计算任务最终都由ALU完成。
- **关键寄存器：**
 - **累加器 (ACC)：**一个临时存储单元，用于存放ALU运算的结果。
 - **标志寄存器 (FLAGS/PSW)：**记录运算的状态，如结果是否为零、是否产生进位或溢出等。
- **控制器 (CU - Control Unit)：**
 - **功能：**CPU的指挥中心。它负责从内存中取出指令（取指），分析指令（译码），然后发出控制信号，命令ALU、存储器等部件执行相应的操作。
 - **关键寄存器：**
 - **程序计数器 (PC)：**存放下一条将要执行的指令的内存地址。PC的内容会自动递增，指向下一条指令，遇到跳转指令时则会被修改为目标地址。
 - **指令寄存器 (IR)：**存放当前正在执行的指令。
- **寄存器组 (Registers)：**
 - **功能：**CPU内部的高速存储单元，速度远快于内存。用于临时存放指令、数据和地址。
 - **分类：**分为通用寄存器（可由程序员灵活使用）和专用寄存器（有特定用途，如PC、IR）。

1.2 缓存层次 (Cache)

- **背景：**CPU的运算速度极快，而主存（内存RAM）的速度相对慢很多。如果CPU每次都直接从内存读取数据，就会花费大量时间在等待上，如同一个飞快的工人总是在等慢吞吞的仓库送货。Cache就是为了解决这个速度不匹配问题而生。
- **原理：**Cache是位于CPU和主存之间的一块容量小但速度极快的存储器。它会预先加载CPU可能要用到的数据。CPU找数据时，会先看Cache里有没有：
 - **命中 (Hit)：**在Cache中找到了，直接高速读取。
 - **未命中 (Miss)：**Cache里没有，才去访问慢速的主存，并把这次用到的数据块加载到Cache中，以备下次使用。
- **层级结构：**现代CPU通常有多级缓存。
 - **L1 Cache (一级缓存)：**最靠近CPU核心，容量最小（几十KB），速度最快（接近寄存器）。
 - **L2 Cache (二级缓存)：**容量和速度介于L1和L3之间。
 - **L3 Cache (三级缓存)：**容量最大（几MB到几十MB），速度最慢，通常为所有CPU核心共享。

高频考点： 存储器速度排序：寄存器 > L1 Cache > L2 Cache > L3 Cache > 主存(RAM) > 固态硬盘(SSD) > 机械硬盘(HDD)。

2. 存储器系统

2.1 RAM vs ROM

- **RAM (Random Access Memory - 随机存取存储器)**

- **特点：**可读可写，但**断电后数据会丢失**（易失性）。我们平时运行的程序、打开的文件，都加载在RAM中。
- **分类：**
 - **DRAM (动态RAM)：**构成我们电脑主内存条的主要芯片。需要不断刷新来维持数据，速度比SRAM慢。
 - **SRAM (静态RAM)：**只要通电数据就一直保持，速度快，成本高，通常用于制造CPU的Cache。

- **ROM (Read-Only Memory - 只读存储器)**

- **特点：**通常只能读取，不能随意写入，**断电后数据不会丢失**（非易失性）。
- **用途：**用于存放固定的、不希望被修改的程序和数据，例如计算机主板上的**BIOS**程序或一些嵌入式设备的固件。

3. I/O 设备与总线

- **I/O (Input/Output)：**指输入/输出设备，如键盘、鼠标、显示器、网卡、硬盘等。
- **总线 (Bus)：**是连接计算机各个部件（CPU、内存、I/O设备）并为它们提供数据交换通道的“公共汽车”。
 - **数据总线：**传输数据。
 - **地址总线：**指定数据要去的内存地址或I/O端口。
 - **控制总线：**传输控制信号。

4. BIOS/UEFI

- **BIOS (Basic Input/Output System)：**固化在主板ROM芯片上的一个程序。
- **功能：**
 1. **开机自检 (POST)：**检查CPU、内存、硬盘等硬件是否工作正常。
 2. **初始化硬件：**设置和准备硬件，使其处于可用状态。
 3. **引导加载程序：**找到硬盘上的操作系统（如Windows, Linux），并把它加载到内存中运行。
- **UEFI：**是BIOS的现代替代品，功能更强大，启动更快，支持更大的硬盘。

模块四 编程语言核心

作战目标：理解我们使用的C++语言的本质，掌握其基本语法规则和常见陷阱，为编写正确的代码打下坚实基础。

1. 语言范式与分类

- **面向过程 (Procedure-Oriented)：**程序由一系列函数（过程）组成，思考方式是“第一步做什么，第二步做什么”。C语言是典型的面向过程语言。

- **面向对象 (Object-Oriented, OOP)**: 程序由一系列“对象”组成，每个对象封装了数据和操作这些数据的方法。C++、Java是典型的面向对象语言。
- **低级语言 vs 高级语言**
 - **低级语言**: 贴近硬件，执行效率高，但编写和移植困难。包括**机器语言**（01代码）和**汇编语言**。
 - **高级语言**: 语法接近自然语言，可读性强，易于移植。C++, Java, Python等都是高级语言。

2. 编译型 vs 解释型

这是高级语言执行方式的两种主要模式。

特性	编译型 (C/C++)	解释型 (Python/JavaScript)
转换过程	源代码 → 编译器 → 目标机器码 → 链接器 → 可执行文件	源代码 → 解释器 → 逐行解释并立即执行
执行速度	快。一次性翻译成机器码，之后可直接运行。	慢。每次执行都需要实时翻译。
可移植性	差。需要为不同平台（Windows/Linux）分别编译。	好。只要有对应平台的解释器，源码无需修改即可运行。
调试	相对复杂，修改后需重新编译。	简单，可即时反馈错误。

高频考点： C++是编译型语言， Python是解释型语言。编译过程会生成目标文件（.o 或 .obj ）。

3. C++ 基本类型与陷阱

- **bool**： 虽然逻辑上只表示 true 或 false （1位），但在内存中通常占用**1个字节 (8位)**，以方便内存寻址。
- **char**： 占用1个字节。 **signed char** 范围是-128到127， **unsigned char** 范围是0到255。
- **int**： 在32位系统中通常为4字节， 范围约

-2×10^9

到

2×10^9

。竞赛中遇到超过

10^9

的数据，要考虑使用 **long long** 。

- **整型溢出**： 当计算结果超出数据类型的表示范围时，会发生溢出。例如，一个 **int** 变量存储了最大值，再加1会变成一个很小的负数。

- **浮点误差**：由于计算机用二进制表示小数，很多十进制小数无法被精确表示，从而产生精度误差。
 - **陷阱**：永远不要用 `==` 直接比较两个浮点数是否相等。
 - **正确做法**：判断它们的差的绝对值是否小于一个极小的数（称为epsilon, 如 $1e-7$ ）。

$$|a - b| < \epsilon$$

4. 指针与数组

4.1 指针基础

- **指针**：一个存储内存地址的变量。
- **数组名**：在大多数表达式中，数组名会被隐式转换成指向数组首元素的指针。

```
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* p; // 声明一个整型指针p

    p = arr; // p指向数组的第一个元素arr[0]
             // 这等价于 p = &arr[0];

    // 访问数组元素
    // *p 的值是 10 (arr[0])
    // *(p + 1) 的值是 20 (arr[1])

    // 修改数组元素
    *(p + 2) = 99; // 将arr[2]的值修改为99

    // 遍历数组
    for (int i = 0; i < 5; i = i + 1) {
        // 使用 *(p + i) 访问元素
        std::cout << *(p + i) << " ";
    }
    std::cout << std::endl;
    // 输出: 10 20 99 40 50

    return 0;
}
```

4.2 二维数组与地址计算

- **内存布局**：二维数组在内存中是**行主序**连续存储的。即第一行所有元素存完后，紧接着存第二行的所有元素，以此类推。
- **地址计算公式**：对于数组 `int a[M][N]`，元素 `a[i][j]` 的地址可以表示为：

$$\text{Address}(a[i][j]) = \text{BaseAddress} + (i \times N + j) \times \text{sizeof}(\text{int})$$

其中 `BaseAddress` 是数组首元素 `a[0][0]` 的地址。

高频考点： 给定一个二维数组的定义和某个元素的地址，求另一个元素的地址，或求数组的维度。

模块五 数据结构精要

作战目标： 掌握信息学竞赛中最核心的“积木块”——数据结构。选择合适的数据结构是解决问题的关键第一步。

1. 线性结构

- **数组 (Array):**
 - 优点：内存连续，支持 **$O(1)$** 复杂度的随机访问（通过下标直接定位）。
 - 缺点：插入和删除元素需要移动大量后续元素，时间复杂度为 **$O(n)$** 。
- **链表 (Linked List):**
 - 优点：插入和删除元素只需修改指针，时间复杂度为 **$O(1)$** （若已知待操作位置的前驱）。
 - 缺点：内存不连续，不支持随机访问，查找某个元素需要从头遍历，时间复杂度为 **$O(n)$** 。
- **栈 (Stack):**
 - 特点：**后进先出 (LIFO - Last In, First Out)**。想象一个羽毛球筒，最后放进去的球最先被拿出。
 - 操作：`push`（入栈）、`pop`（出栈）。
 - 应用：函数调用（递归的内部实现）、表达式求值、括号匹配、深度优先搜索 (DFS)。
- **队列 (Queue):**
 - 特点：**先进先出 (FIFO - First In, First Out)**。就像排队买票，先来的人先买。
 - 操作：`push`（入队）、`pop`（出队）。
 - 应用：广度优先搜索(BFS)、任务调度系统。

2. 树结构

2.1 二叉树

- 定义：每个节点最多有两个子节点，分别称为左子节点和右子节点。
- 特殊形态：
 - 满二叉树：一个深度为 k 的树，拥有

$$2^k - 1$$

个节点，每一层都是满的。

- **完全二叉树**：除了最后一层，其他层都是满的，且最后一层的节点都连续集中在左侧。堆就是一种完全二叉树。
- **重要性质 (高频考点)**:
 1. 深度为h的二叉树最多有

$$2^h - 1$$

个节点。

2. 对于任意二叉树，度为0的节点（叶子节点）数比度为2的节点数多1，即

$$n_0 = n_2 + 1$$

。

3. 对于一个有n个节点的**完全二叉树**，若节点从1开始编号，则节点i的：

- 父节点编号：

$$i/2$$

(整除)

- 左子节点编号：

$$2 \times i$$

- 右子节点编号：

$$2 \times i + 1$$

2.2 遍历

- **先序遍历 (根-左-右)**：常用于树的复制。
- **中序遍历 (左-根-右)**：对于二叉搜索树，中序遍历结果是一个有序序列。
- **后序遍历 (左-右-根)**：常用于计算依赖子树结果的表达式树。
- **由遍历序列重构二叉树 (高频考点)**:
 - **(先序 + 中序)** 或 **(后序 + 中序)** 可以**唯一确定**一棵二叉树。
 - **(先序 + 后序)** 不能唯一确定一棵二叉树（除非是满二叉树）。
 - **重构技巧**:
 1. 在先序序列中找到根节点（第一个元素）。
 2. 在中序序列中找到该根节点，其左边的序列是左子树的中序遍历，右边是右子树的中序遍历。
 3. 根据中序序列中左右子树的长度，可以在先序序列中划分出左右子树的先序遍历序列。
 4. 递归地对左右子树进行上述过程。

3. 图结构

- **图的表示法**:
 - **邻接矩阵**：用一个二维数组 $G[N][N]$ 存储， $G[i][j]=1$ 表示i到j有边。

- 优点：判断两点间是否有边为 $O(1)$ 。
- 缺点：空间复杂度为 $O(N^2)$ ，对于稀疏图（边少）浪费空间。
- 邻接表：用一个数组，每个元素挂一个链表（或 vector），存储从该点出发的所有边。
 - 优点：空间复杂度为 $O(N+E)$ ，节省空间。
 - 缺点：判断两点间是否有边需要 $O(k)$ （ k 为出度）。
- 图的遍历：
 - DFS (深度优先搜索)：利用递归或栈，一条路走到黑，再回溯。
 - BFS (广度优先搜索)：利用队列，一层一层地向外扩展。在无权图中，BFS可以找到最短路径。
- 重要图论算法概念：
 - 拓扑排序：对一个有向无环图 (DAG) 的顶点进行排序，使得所有边都从排序较前的顶点指向较后的顶点。
 - 最短路径：
 - Dijkstra：用于无负权边的单源最短路径。
 - Floyd：用于多源最短路径，可以处理负权边（但不能有负权环）。
 - 最小生成树 (MST)：在一个连通的无向图中，找到一个权值总和最小的边的子集，使得所有顶点都连通。
 - Prim：从一个点开始，不断选择离当前集合最近的点加入。
 - Kruskal：按边权从小到大排序，依次加入不构成环的边。

模块六 算法专项突破

作战目标：掌握常用算法的设计思想和实现模板，将理论知识转化为解题能力。

1. 排序算法全解

排序方法	稳定性	平均时间复杂度	最坏时间复杂度	空间复杂度	备注
冒泡排序	稳定	$O(n^2)$	$O(n^2)$	$O(1)$	简单，但效率低
选择排序	不稳定	$O(n^2)$	$O(n^2)$	$O(1)$	交换次数少
插入排序	稳定	$O(n^2)$	$O(n^2)$	$O(1)$	对近乎有序的数据效率高
归并排序	稳定	$O(n \log n)$	$O(n \log n)$	$O(n)$	分治思想，空间换时间
快速排序	不稳定	$O(n \log n)$	$O(n^2)$	$O(\log n)$	综合性能最好，但最坏情况差

排序方法	稳定性	平均时间复杂度	最坏时间复杂度	空间复杂度	备注
堆排序	不稳定	$O(n \log n)$	$O(n \log n)$	$O(1)$	利用堆数据结构
计数排序	稳定	$O(n + k)$	$O(n + k)$	$O(k)$	k为数据范围，适用于整数
桶排序	稳定	$O(n + k)$	$O(n^2)$	$O(n + k)$	计数排序的推广
基数排序	稳定	$O(d(n+r))$	$O(d(n+r))$	$O(n+r)$	按位排序，d为位数，r为基数

稳定性解释：如果数组中两个相等的元素，在排序后它们的相对位置不变，则该排序算法是稳定的。例如，对成绩和姓名排序，希望成绩相同时，原来靠前的学生仍然靠前。

高频考点：

- 1. 各种排序算法的时间/空间复杂度和稳定性是必考内容。
- 2. 快速排序在数据已有序或逆序时，会退化到 $O(n^2)$ 。
- 3. 对于整数范围不大（如0-1000）的排序，计数排序是最佳选择。

2. 二分查找

- **前提：**待查找的序列必须是**有序**的。
- **思想：**每次取中间位置的元素与目标值比较，根据比较结果将查找范围缩小一半。
- **标准模板：**

```
// 在升序数组 a 中查找 target
int binary_search(int a[], int n, int target) {
    int left = 0;
    int right = n - 1; // 闭区间 [left, right]
    int mid;

    while (left <= right) {
        mid = left + (right - left) / 2; // 防止 left+right 溢出
        if (a[mid] == target) {
            return mid; // 找到了，返回下标
        } else if (a[mid] < target) {
            left = mid + 1; // 目标在右半部分
        } else {
            right = mid - 1; // 目标在左半部分
        }
    }
    return -1; // 没找到
}
```

高频考点： 二分查找的变种，如查找第一个大于等于 `target` 的元素（`lower_bound`）或第一个大于 `target` 的元素（`upper_bound`）。这通常通过调整 `left` 和 `right` 的更新逻辑和循环条件来实现。

3. 其他经典算法思想

- **分治 (Divide and Conquer):** 将大问题分解为若干个相同或相似的子问题，递归地解决子问题，再合并子问题的解。
 - 例子：归并排序、快速排序、二分查找。
- **贪心 (Greedy):** 在每一步选择中都采取当前状态下最好或最优的选择，从而希望导致全局结果是最好或最优的。
 - 特点：简单高效，但不能保证得到全局最优解。
 - 例子：区间调度、霍夫曼编码、Prim和Kruskal算法。
- **动态规划 (Dynamic Programming, DP):** 通过把原问题分解为相对简单的子问题的方式求解复杂问题。通常用于求解最优化问题。
 - 核心：状态定义 和 状态转移方程。
 - 特征：最优子结构、重叠子问题。
 - 例子：背包问题、最长公共子序列(LCS)、Floyd算法。

模块七 竞赛技巧与题目练习

作战目标： 将所有知识融会贯通，掌握初赛的答题策略和常见题型，并进行实战演练。

1. 阅读程序题策略

1. **快速判断算法类型：** 扫视代码，识别关键结构（如排序循环、递归、DFS/BFS模板），初步判定程序意图。
2. **静态分析变量作用：** 标记循环变量、累加器、计数器、状态标志等核心变量，理解它们的功能。
3. **手动模拟执行流程：** 代入一个简单的、小规模输入（如 $n=3$ 或 $n=4$ ），手动模拟程序的执行过程，跟踪关键变量的变化。这是最有效的方法。
4. **关注边界和特殊情况：** 思考输入为0、1、最大值，或数据全部相等、逆序等情况时，程序的行为。

2. 补全程序题策略

这类题通常是考察对经典算法实现的熟悉程度。

- **高频考点：**
 - **二分查找：** `mid` 的计算、`left` 和 `right` 的更新。
 - **DFS/BFS：** 递归的终止条件、队列/栈的操作、`visited` 数组的标记。
 - **动态规划：** 状态转移方程的核心部分。

- 排序算法：partition 中的交换逻辑、归并排序的合并逻辑。
- 图论：Dijkstra 中松弛操作的条件。

3. 复杂度分析

3.1 时间复杂度

- 循环：单层循环为 $O(n)$ ，双层嵌套循环为 $O(n^2)$ 。
- 递归：使用主定理或画递归树分析。
 - 二分查找：

$$T(n) = T(n/2) + O(1) \implies O(\log n)$$

- 归并排序：

$$T(n) = 2T(n/2) + O(n) \implies O(n \log n)$$

- 常见复杂度排序：

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

3.2 空间复杂度

- 计算程序运行时额外开辟的内存空间。
- 递归：空间复杂度与最大递归深度有关。
- 数组：`int a[1000]` 的空间复杂度是 $O(1000)$ ，即 $O(1)$ ；`int a[n]` 的空间复杂度是 $O(n)$ 。
- 内存限制估算：竞赛中常见的256MB内存，大约可以开一个 `int` 数组的大小为：

$$\frac{256 \times 1024 \times 1024 \text{ (Byte)}}{4 \text{ (Byte/int)}} \approx 6.7 \times 10^7$$

即六千七百万左右。开 `int a[10000][10000]` 是肯定会超空间的。